```
import os, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

FILE_ID = "1_9CDEYJ_8xRoZaCNVA_seGV7RYnxvw7i"
!gdown {FILE_ID} -O /content/data.zip
```

```
Downloading...
From (original): https://drive.google.com/uc?
id=1_9CDEYJ_8xRoZaCNVA_seGV7RYnxvw7i
From (redirected): https://drive.google.com/uc?
id=1_9CDEYJ_8xRoZaCNVA_seGV7RYnxvw7i&confirm=t&uuid=3960c29d-9471-
4470-826a-f667cdefd3da
To: /content/data.zip
100% 2.15G/2.15G [00:32<00:00, 66.8MB/s]
```

```
!unzip -q /content/data.zip -d /content/data -x "__MACOSX/*"
!du -sh /content/data
```

```
caution: excluded filename not matched:  __MACOSX/*
2.1G  /content/data
```

```
IMG_EXT = (".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff")
DATA_ROOT = "/content/data"

for root, dirs, files in os.walk(DATA_ROOT):
    dirs[:] = sorted(d for d in dirs if not d.startswith(("__MACOSX",
".")))
    imgs = [f for f in files if f.lower().endswith(IMG_EXT)]
    depth = root[len(DATA_ROOT):].count(os.sep)
    other = len(files) - len(imgs)
    print("  " * depth + f"{os.path.basename(root) or DATA_ROOT}/  ->
{len(imgs)} images" + (f", {other} other files" if other else ""))
```

```
data/  -> 0 images
  Pneumonia X Ray/  -> 0 images
    chest_xray/  -> 0 images, 1 other files
      test/  -> 0 images, 1 other files
        NORMAL/  -> 234 images
        PNEUMONIA/  -> 390 images
      train/  -> 0 images, 1 other files
        NORMAL/  -> 1341 images, 1 other files
        PNEUMONIA/  -> 2954 images
      val/  -> 0 images, 1 other files
        NORMAL/  -> 8 images, 1 other files
        PNEUMONIA/  -> 8 images, 1 other files
    test/  -> 0 images
      NORMAL/  -> 234 images
      PNEUMONIA/  -> 390 images
```

```
    train/  -> 0 images
      NORMAL/  -> 1091 images
      PNEUMONIA/  -> 2846 images
    val/  -> 0 images
      NORMAL/  -> 8 images
      PNEUMONIA/  -> 8 images
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
classification_report

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

import os, random
from PIL import Image

tf.random.set_seed(42)
np.random.seed(42)

IMG_EXT=(".jpg",".jpeg",".png")
CLASSES=["NORMAL","PNEUMONIA"]

TRAIN_DIR="/content/data/Pneumonia X Ray/train"
TEST_DIR="/content/data/Pneumonia X Ray/test"

TRAIN_NORM="/content/data/Pneumonia X Ray/train/NORMAL"
TRAIN_PNEU="/content/data/Pneumonia X Ray/train/PNEUMONIA"
TEST_NORM="/content/data/Pneumonia X Ray/test/NORMAL"
TEST_PNEU="/content/data/Pneumonia X Ray/test/PNEUMONIA"

n_train_normal=len(os.listdir(TRAIN_NORM))
n_train_pneu=len(os.listdir(TRAIN_PNEU))
n_test_normal=len(os.listdir(TEST_NORM))
n_test_pneu=len(os.listdir(TEST_PNEU))

print("train/NORMAL:   ", n_train_normal)    # expect 1091
print("train/PNEUMONIA:", n_train_pneu)      # expect 2846
print("test/NORMAL:    ", n_test_normal)     # expect 234
print("test/PNEUMONIA: ", n_test_pneu)

print("GPU:", tf.config.list_physical_devices("GPU"))
```

```
#Jupyter version
# import os, random
# import numpy as np
# import pandas as pd
# import matplotlib.pyplot as plt
# from PIL import Image
# import tensorflow as tf
# from tensorflow import keras
# from tensorflow.keras import layers
# from tensorflow.keras.optimizers import Adam
# from tensorflow.keras.callbacks import EarlyStopping
# from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay

# tf.random.set_seed(42)
# random.seed(42)

# IMG_EXT = (".jpg", ".jpeg", ".png", ".bmp")
# CLASSES = ["NORMAL", "PNEUMONIA"]       # must match the class
folder names exactly

# DATA = r"C:\Users\student\Desktop\Pneumonia X Ray"      # <-- the
folder that contains train, test, val
# print(os.listdir(DATA))                                  # check the
folder names before going on

# # folders
# TRAIN_DIR = os.path.join(DATA, "train")
# TEST_DIR = os.path.join(DATA, "test")

# TRAIN_NORMAL = os.path.join(DATA, "train", "NORMAL")
# TRAIN_PNEU = os.path.join(DATA, "train", "PNEUMONIA")
# TEST_NORMAL = os.path.join(DATA, "test", "NORMAL")
# TEST_PNEU = os.path.join(DATA, "test", "PNEUMONIA")

# # counts, one folder per line
# n_train_normal = len(os.listdir(TRAIN_NORMAL))
# n_train_pneu = len(os.listdir(TRAIN_PNEU))
# n_test_normal = len(os.listdir(TEST_NORMAL))
# n_test_pneu = len(os.listdir(TEST_PNEU))

# print("train/NORMAL:   ", n_train_normal)
# print("train/PNEUMONIA:", n_train_pneu)
# print("test/NORMAL:    ", n_test_normal)
# print("test/PNEUMONIA: ", n_test_pneu)
# print("GPU:", tf.config.list_physical_devices("GPU"))

train/NORMAL:    1091
train/PNEUMONIA: 2846
test/NORMAL:     234
```

```
test/PNEUMONIA:  390
GPU: []

normal_files = random.sample([f for f in os.listdir(TRAIN_NORM) if
f.lower().endswith(IMG_EXT)], 4)
pneu_files=random.sample([f for f in os.listdir(TRAIN_PNEU) if
f.lower().endswith(IMG_EXT)],4)

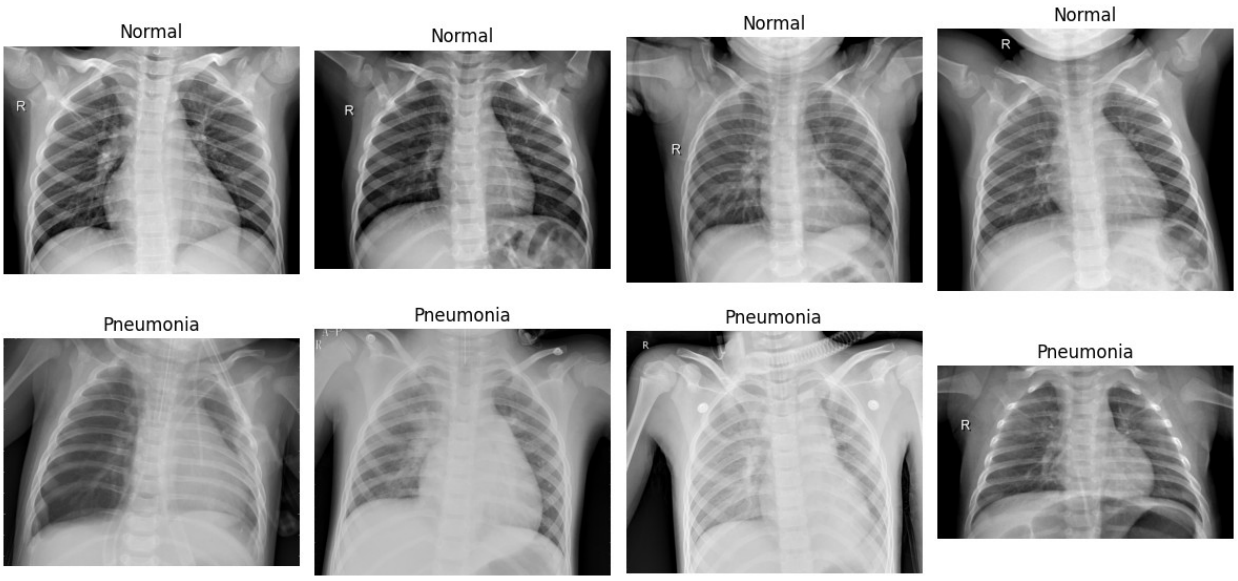
fig,axes=plt.subplots(2,4,figsize=(12,6))

for k in range(4):

axes[0,k].imshow(Image.open(os.path.join(TRAIN_NORM,normal_files[k])),
cmap="gray")
  axes[0,k].set_title("Normal")
  axes[0,k].axis("off")

for k in range(4):
  axes[1,k].imshow(Image.open(os.path.join(TRAIN_PNEU,
pneu_files[k])),cmap="gray")
  axes[1,k].set_title("Pneumonia")
  axes[1,k].axis("off")

plt.tight_layout()
plt.show()
```




```
IMG_SIZE=(128,128)
BATCH=32

train_ds=keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
```

```
    label_mode="binary",
    color_mode="grayscale",
    image_size=IMG_SIZE,
    batch_size=BATCH,
    validation_split=0.2,
    subset="training",
    seed=42
)
val_ds=keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    label_mode="binary",
    color_mode="grayscale",
    image_size=IMG_SIZE,
    batch_size=BATCH,
    validation_split=0.2,
    subset="validation",
    seed=42
)
test_ds = keras.utils.image_dataset_from_directory(
    TEST_DIR,
    label_mode="binary",
    color_mode="grayscale",
    image_size=IMG_SIZE,
    batch_size=BATCH,
    shuffle=False
)
```

```
Found 3937 files belonging to 2 classes.
Using 3150 files for training.
Found 3937 files belonging to 2 classes.
Using 787 files for validation.
Found 624 files belonging to 2 classes.
```

```
n0=n_train_normal
n1=n_train_pneu
total=n0+n1

class_weights={0: total/(2*n0), 1:total/(2*n1)}
print(class_weights)
```

```
{0: 1.804307974335472, 1: 0.6916725228390724}
```

```
model=keras.Sequential([
    layers.Input(shape=(128,128,1)),
    layers.Rescaling(1.0/255.0),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),

    layers.Conv2D(32,(3,3), activation="relu"),
    layers.MaxPooling2D((2,2)),
```

```
    layers.Conv2D(64,(3,3),activation="relu"),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(128,(3,3),activation="relu"),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(128,activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1,activation="sigmoid")
])

model.summary()
```

```
Model: "sequential"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ rescaling_1 (Rescaling)              │ (None, 128, 128, 1)         │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ random_rotation_1                    │ (None, 128, 128, 1)         │               0 │
│ (RandomRotation)                     │                             │                 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ random_zoom_1 (RandomZoom)           │ (None, 128, 128, 1)         │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ conv2d_2 (Conv2D)                    │ (None, 126, 126, 32)        │             320 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ max_pooling2d_1 (MaxPooling2D)       │ (None, 63, 63, 32)          │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ conv2d_3 (Conv2D)                    │ (None, 61, 61, 64)          │          18,496 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ max_pooling2d_2 (MaxPooling2D)       │ (None, 30, 30, 64)          │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
```

```
──────┤
│ conv2d_4 (Conv2D)                   │ (None, 28, 28, 128)     │
73,856 │
├─────────────────────────────────────┼─────────────────────────┼──────
──────┤
│ max_pooling2d_3 (MaxPooling2D)      │ (None, 14, 14, 128)     │
0 │
├─────────────────────────────────────┼─────────────────────────┼──────
──────┤
│ flatten (Flatten)                   │ (None, 25088)           │
0 │
├─────────────────────────────────────┼─────────────────────────┼──────
──────┤
│ dense (Dense)                       │ (None, 128)             │
3,211,392 │
├─────────────────────────────────────┼─────────────────────────┼──────
──────┤
│ dropout (Dropout)                   │ (None, 128)             │
0 │
├─────────────────────────────────────┼─────────────────────────┼──────
──────┤
│ dense_1 (Dense)                     │ (None, 1)               │
129 │
└─────────────────────────────────────┴─────────────────────────┴──────
──────┘

 Total params: 3,304,193 (12.60 MB)

 Trainable params: 3,304,193 (12.60 MB)

 Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    loss="binary_crossentropy",
    optimizer=Adam(learning_rate=0.001),
    metrics=["accuracy"]
)

early=EarlyStopping(monitor="val_loss",patience=4,restore_best_weights
=True)

history=model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=7,
    class_weight=class_weights,
    callbacks=[early],
    verbose=2
)
```

```
Epoch 1/7
99/99 - 141s - 1s/step - accuracy: 0.9298 - loss: 0.1781 -
val_accuracy: 0.9161 - val_loss: 0.2304
Epoch 2/7
99/99 - 137s - 1s/step - accuracy: 0.9403 - loss: 0.1545 -
val_accuracy: 0.9072 - val_loss: 0.2411
Epoch 3/7
99/99 - 131s - 1s/step - accuracy: 0.9432 - loss: 0.1563 -
val_accuracy: 0.9161 - val_loss: 0.2386
Epoch 4/7
99/99 - 129s - 1s/step - accuracy: 0.9460 - loss: 0.1496 -
val_accuracy: 0.9670 - val_loss: 0.0916
Epoch 5/7
99/99 - 128s - 1s/step - accuracy: 0.9540 - loss: 0.1281 -
val_accuracy: 0.9416 - val_loss: 0.1589
Epoch 6/7
99/99 - 138s - 1s/step - accuracy: 0.9600 - loss: 0.1101 -
val_accuracy: 0.9593 - val_loss: 0.0999
Epoch 7/7
99/99 - 129s - 1s/step - accuracy: 0.9603 - loss: 0.0977 -
val_accuracy: 0.9543 - val_loss: 0.1095

test_loss, test_acc=model.evaluate(test_ds)
print("Test loss:", test_loss)
print("Test accuracy:", test_acc)

20/20 ━━━━━━━━━━━━━━━━━━━━ 9s 399ms/step - accuracy: 0.8253 - loss:
0.4655
Test loss: 0.4654558598995209
Test accuracy: 0.8253205418586731

y_true=np.concatenate([y.numpy() for x,y in
test_ds]).ravel().astype(int)
y_prob=model.predict(test_ds).ravel()
y_pred = (y_prob > 0.5).astype(int)

print(classification_report(y_true, y_pred, target_names=CLASSES))
print(confusion_matrix(y_true, y_pred))
ConfusionMatrixDisplay.from_predictions(y_true, y_pred,
display_labels=CLASSES)
plt.show()

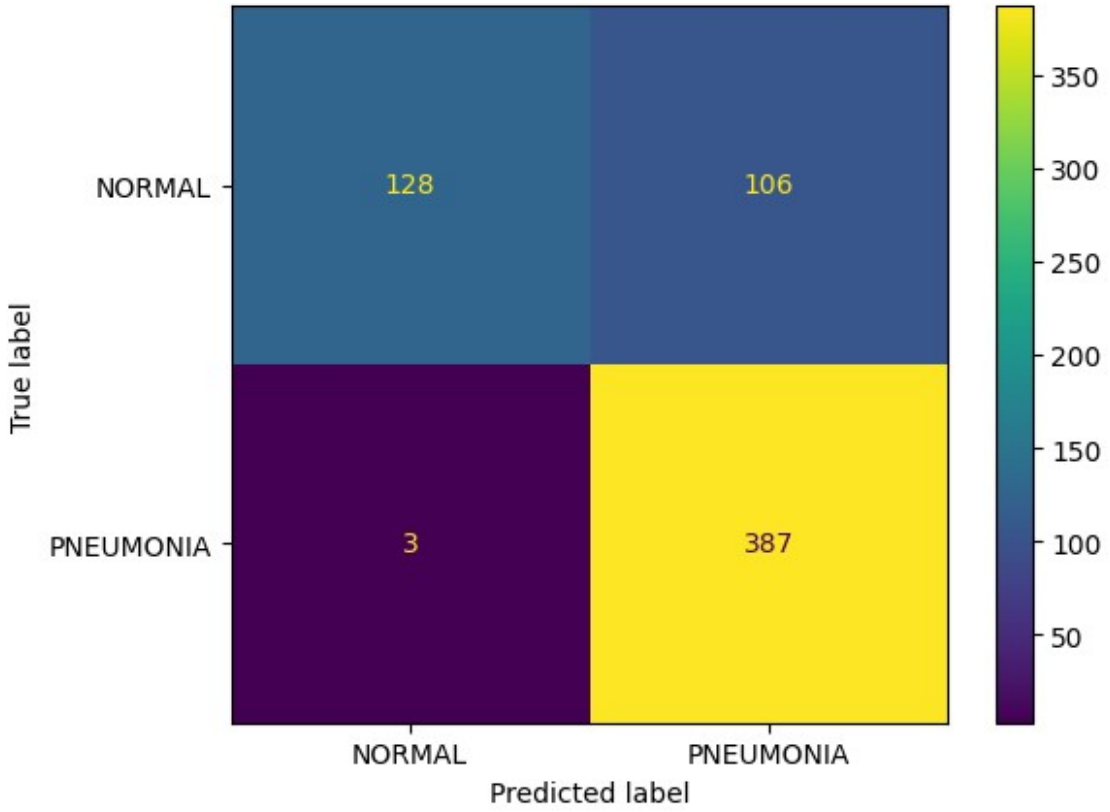
20/20 ━━━━━━━━━━━━━━━━━━━━ 13s 581ms/step
              precision    recall  f1-score   support

      NORMAL       0.98      0.55      0.70       234
   PNEUMONIA       0.78      0.99      0.88       390

    accuracy                           0.83       624
   macro avg       0.88      0.77      0.79       624
```

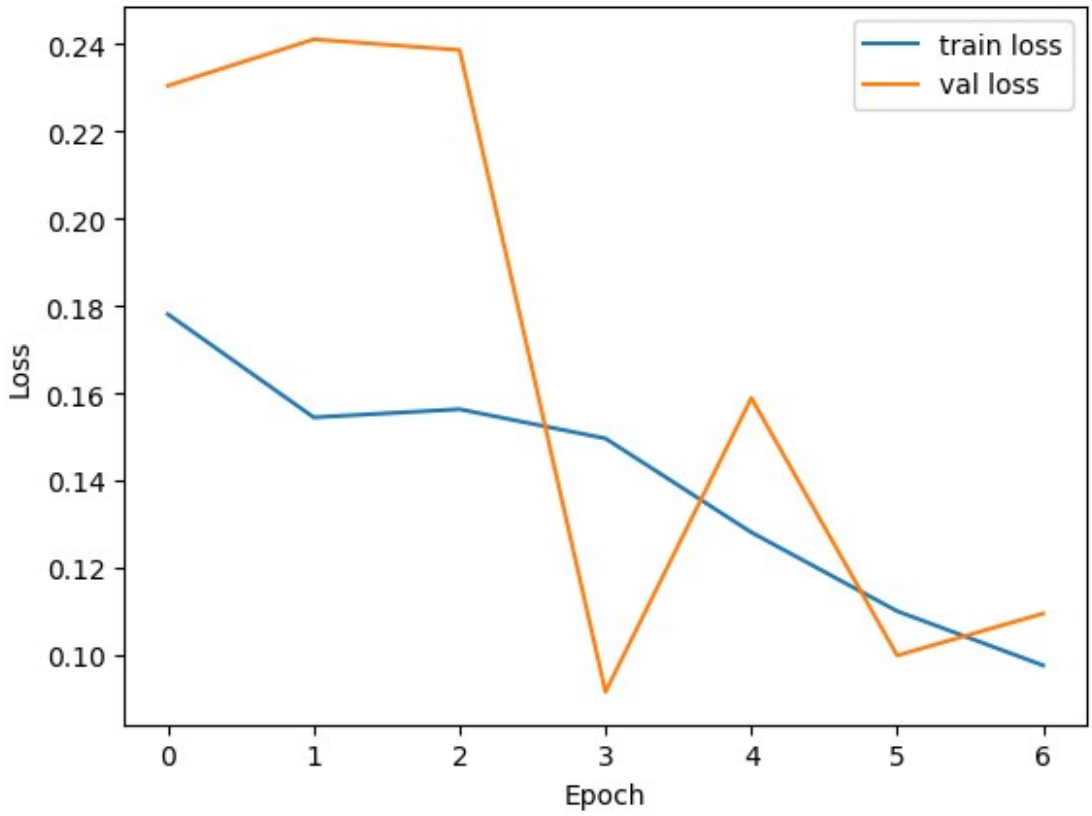
```
weighted avg       0.86      0.83      0.81       624

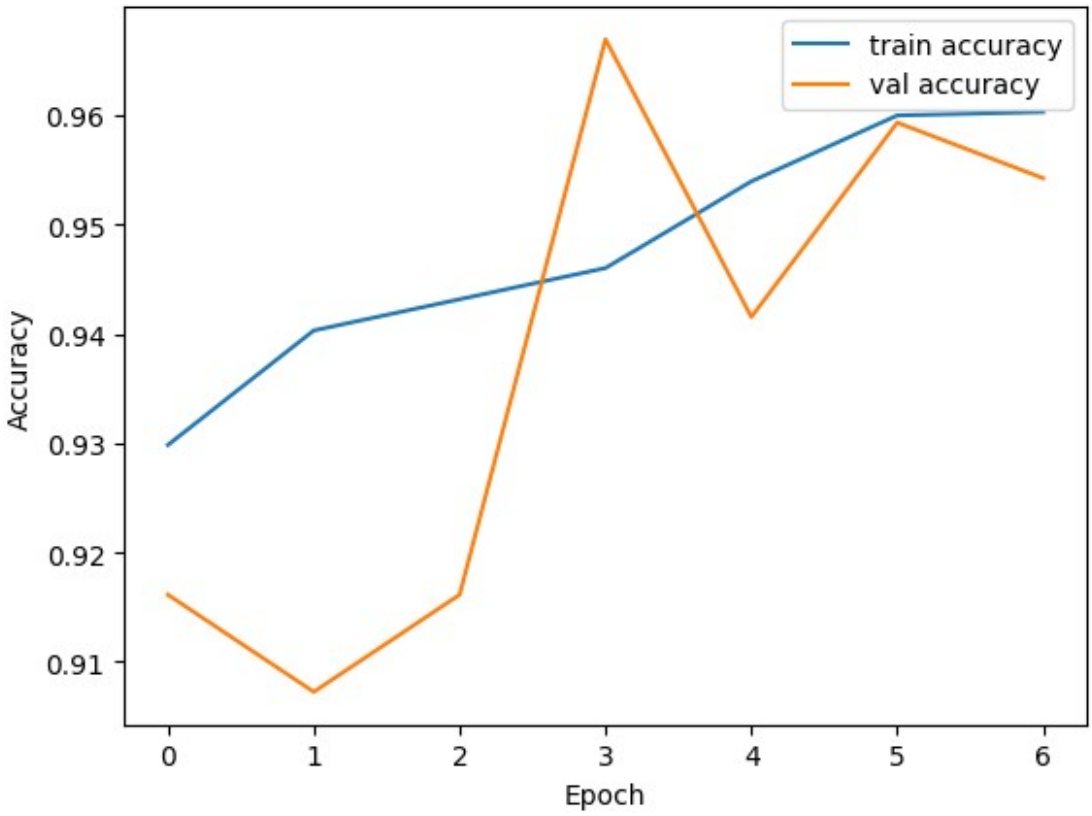
[[128 106]
 [  3 387]]
```




```
plt.plot(history.history["loss"], label="train loss")
plt.plot(history.history["val_loss"], label="val loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()

plt.plot(history.history["accuracy"], label="train accuracy")
plt.plot(history.history["val_accuracy"], label="val accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


train loss
val loss
Loss
Epoch
0.24
0.22
0.20
0.18
0.16
0.14
0.12
0.10
0
1
2
3
4
5
6


train accuracy
val accuracy
Accuracy
Epoch
0.96
0.95
0.94
0.93
0.92
0.91
0
1
2
3
4
5
6